TU Dortmund

Fachprojekt Detektion von Sicherheitslücken

M.Sc. Johannes Düsing, M.Sc. Anemone Kampkötter

JProf. Dr.-Ing. Ben Hermann

Aufgabe 4: Manipulation von Bytecode

Ausgabe: 20.11.2024

Besprechung: 04.12.2024

Die *Java Virtual Machine* (JVM) führt Programme aus, die als sogenannter *Java Bytecode* vorliegen. Der Bytecode ist eine Stack-basierte Zwischensprache und wird normalerweise von Compilern erzeugt. Informationen zu den einzelnen Instruktionen und deren Effekten finden sich in der offiziellen Spezifikation[1].

Der Bytecode wird von Compilern als `.class` Dateien ausgegeben und in der Regel in einem `.jar` Archiv gebündelt. Es ist allerdings auch ohne weiteres möglich, Bytecode *von Hand* zu verfassen bzw. vorhandenen Bytecode abzuändern. Beides sollen Sie in dieser Aufgabe tun.

4.1 Analyseaufgabe

4.1.1 Bytecode Modifizieren

Für Analyseaufgabe 2 haben Sie das Aufrufen von verbotenen Methoden detektiert und auf der Konsole ausgegeben. In dieser Teilaufgabe sollen Sie diese Implementation als Grundlage nutzen und schrittweise erweitern. Ihre Analyse soll:

1. Aufrufe bestimmter, von Nutzenden konfigurierbarer Methoden detektieren (wie schon bei Analayseaufgabe 2).
2. Die gesamten Bytecode Instruktionen der Methode, welche die verbotene Methode aufruft, auf der Kommandozeile ausgeben.
3. Den verbotenen Aufruf aus dem Bytecode entfernen und eine Kopie der entsprechenden `.class` Datei anlegen, in der die verbotenen Aufrufe nicht mehr vorhanden sind.

JVM Bytecode ist eine Stack-Basierte Sprache. Das heißt, Methodenaufrufe erwarten bestimmte Werte (unter anderem die Parameter) auf dem Stack, und legen unter Umständen einen Rückgabewert zurück auf den Stack. Informieren Sie sich in der Sprach-Spezifikation darüber, welche Stack-Layouts die vier verschiedenen Aufruf-Instruktionen (`INVOKEVIRTUAL`, `INVOKESTATIC`, `INVOKESPECIAL` und `INVOKEDYNAMIC`) erwarten. Präsentieren Sie uns anschließend, ob das Entfernen der Aufrufe in Ihrer Implementation Auswirkungen auf das Stack-Layout hat, ob (und wenn ja, wie) Sie diese Auswirkungen adressiert haben, und wie man mit dem Problem theoretisch umgehen könnte.

4.1.2 Bytecode Erstellen

In Analyseaufgabe 3 haben Sie Aufrufe in Third Party Libraries (TPLs) detektiert. Für diese Teilaufgabe sollen Sie darauf aufbauen. Ihre Analyse soll für **eine TPL** eines Softwaresystems einen sogenannten *Dummy* generieren. Der Dummy soll **nur genau die** Klassen und Methoden aus der originalen TPL beinhalten, die im Kontext des Softwaresystems aufgerufen bzw. genutzt werden. Alle anderen Klassen und Methoden sollen nicht vorhanden sein. Die Methoden sollen keinen tatsächlichen Bytecode beinhalten, sondern lediglich `null` zurückgeben, falls ein Rückgabewert gefordert ist.

[1] `https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html#jvms-6.5`

Die Analyse erhält als Eingabe das Softwaresystem (Projekt-Code und TPL Code) sowie den Namen **einer der TPLs**, und produziert eine Menge von `.class` Dateien, welche die Dummy-TPL darstellen.

Hinweise:

- Das Kommandozeilenwerkzeug `javap` ist in jeder Java-Installation enthalten und kann genutzt werden um den Bytecode von `.class` Dateien zu inspizieren.
- Auf der OPAL Webseite steht ein Tutorial zum Erstellen von Bytecode bereit[2].

Weitere Anforderungen:

- Stellen Sie sicher, dass Ihre Implementierung die geforderte Funktionalität korrekt umsetzt. Schreiben Sie mindestens einen Unit-Test[3] für einen Teil Ihres Codes und messen sie die Testabdeckung (Coverage).
- Stellen Sie uns Ihre Lösung wieder in Ihrem Git-Repository zur Verfügung.

[2] `https://www.opal-project.de/examples/BytecodeEngineering.html`
[3] `https://www.baeldung.com/scala/scalatest`